Ch 6:Basic SQL

Date: 23.7.2026

Lecture 2

What is SQL?

SQL (Structured Query Language) is a programming language specially designed to work with **Relational Database Management Systems (RDBMS)** such as **MySQL, Oracle, SQL Server, PostgreSQL**, etc.

SQL is used to:

- Store data in databases
- Retrieve data from databases
- Modify existing data
- Create and manage database structures

SQL commands are divided into different groups based on their purposes.

Data Query Group (DQL - Data Query Language)

- Used to **retrieve (search) data** from database tables.
- The main command is **SELECT**
- **SELECT** is used to retrieve data from one or more tables.

Syntax:

SELECT column_name

FROM table_name; -- **FROM** specifies the table from which data is taken.

SELECT Fname, Minit, Lname, Salary

FROM Employee;

JOIN

JOIN is used to combine data from multiple tables based on related columns.

- **EMPLOYEE** (Fname, Minit, Lname, **Ssn**, Bdate, Address, Sex, Salary, Super_ssn, **Dno**)
- **DEPARTMENT** (Dname, **Dnumber**, Mgr_ssn, Mgr_start_date)
- **PROJECT** (Pname, **Pnumber**, Plocation, **Dnum**)
- **WORKS_ON** (Essn, **Pno**, Hours)
- **DEPENDENT** (**Essn**, Dependent_name, Sex, Bdate, Relationship)

```
SELECT Employee.Fname, Department.Dname
FROM Employee JOIN Department
ON Employee.Dno = Department.Dnumber;
```

WHERE is used to specify conditions.

Example:

```
SELECT *
FROM Employee
WHERE Salary > 50000;
```

Display employees whose salary is greater than 50000.

GROUP BY combines rows having the same values.

Usually used with aggregate functions:

- COUNT()
- SUM()
- AVG()
- MAX()
- MIN()

```
SELECT Dno, AVG(Salary)
FROM Employee
GROUP BY Dno;
```

HAVING filters groups created by GROUP BY.

```
SELECT Dept_ID, AVG(Salary)
FROM Employee
GROUP BY Dept_ID
HAVING AVG(Salary)>60000;
```

Show departments where average salary is greater than 60000.

ORDER BY sorts the result.

Ascending:

```
SELECT *
FROM Employee
ORDER BY Salary ASC;
```

Descending:

```
SELECT *
FROM Employee
ORDER BY Salary DESC;
```

LIMIT clause is used in SQL to **restrict the number of rows returned** by a query.

Syntax

```
SELECT column_name
FROM table_name
LIMIT number;
```

Eg 1. Display First 3 Employees

```
SELECT *
FROM EMPLOYEE
LIMIT 3;
```

Eg 2: Find Top 3 Highest Salary Employees

```
SELECT Fname, Lname, Salary
FROM EMPLOYEE
ORDER BY Salary DESC
LIMIT 3;
```

LIMIT with OFFSET

Syntax

```
SELECT column_name
FROM table_name
LIMIT offset, count;// LIMIT count OFFSET offset;
```

```
SELECT *
FROM EMPLOYEE
LIMIT 2,3;
```

Eg 3: Find first 2 employees who work in Department 5.

```
SELECT Fname, Lname, Salary
FROM EMPLOYEE
WHERE Dno = 5
LIMIT 2;
```

Data Manipulation Group (DML - Data Manipulation Language)

- Used to insert, update, and delete data inside tables

INSERT is used to add **new rows (records)** into a table.

Syntax 1: Insert Values into All Columns

```
INSERT INTO table_name
VALUES (value1, value2, value3, ...);
```

Syntax 2: Insert Values into Specific Columns

```
INSERT INTO table_name(column1, column2, ...)
VALUES(value1, value2, ...);
```

Syntax 3: Insert Multiple Rows

```
INSERT INTO table_name
VALUES(value1,value2),
(value3,value4);
```

UPDATE is used to modify existing data in a table.

Syntax:

```
UPDATE table_name
SET column1=value1, column2=value2
WHERE condition;
```

```
UPDATE Student
SET Age=21
WHERE Student_ID=1;
```

```
UPDATE Student
SET Name='Aung Aung', Age=22
WHERE Student_ID=1;
```

MERGE combines **INSERT and UPDATE.**

It checks whether data exists:

- If matched → UPDATE
- If not matched → INSERT

Standard SQL MERGE Syntax:

```
MERGE INTO target_table AS T
USING source_table AS S ON condition
WHEN MATCHED THEN UPDATE
   SET column=value
WHEN NOT MATCHED THEN
   INSERT(columns)
   VALUES(values);
```

```
MERGE INTO Employee E
USING New_Employee N
ON E.ID = N.ID
WHEN MATCHED THEN
UPDATE
SET E.Salary=N.Salary
WHEN NOT MATCHED THEN
INSERT(ID,Name,Salary)
VALUES(N.ID,N.Name,N.Salary);
```

MySQL does not support the standard MERGE statement directly.

MySQL usually uses:

INSERT ... ON DUPLICATE KEY UPDATE

DELETE removes existing rows from a table.

Syntax:

```
DELETE *
FROM table_name
WHERE condition;
```

Example:

Delete student with ID 5:

```
DELETE *
FROM Student
WHERE Student_ID=5;
```

No	Command	Syntax	Example (Company Database)
1	SHOW DATABASES	SHOW DATABASES;	SHOW DATABASES;
2	USE Database	USE database_name;	USE company;
3	SHOW TABLES	SHOW TABLES;	SHOW TABLES;
4	SHOW COLUMNS	SHOW COLUMNS FROM table_name;	SHOW COLUMNS FROM EMPLOYEE;
5	DESC	DESC table_name;	DESC EMPLOYEE;
6	SHOW CREATE DATABASE	SHOW CREATE DATABASE database_name;	SHOW CREATE DATABASE company;
7	SHOW CREATE TABLE	SHOW CREATE TABLE table_name;	SHOW CREATE TABLE EMPLOYEE;
8	SHOW INDEX	SHOW INDEX FROM table_name;	SHOW INDEX FROM EMPLOYEE;
9	SHOW GRANTS	SHOW GRANTS FOR user;	SHOW GRANTS FOR 'root'@'localhost';
10	SHOW VARIABLES	SHOW VARIABLES;	SHOW VARIABLES LIKE 'port';
11	SHOW STATUS	SHOW STATUS;	SHOW STATUS LIKE 'Threads_connected';

The **LIKE** operator is used in SQL to **search for a specific pattern in a column**.

It is mainly used with **character (string) data** such as names, addresses, and descriptions.

Syntax

```
SELECT column_name
FROM table_name
WHERE column_name
LIKE pattern;
```

LIKE Wildcards

SQL uses two main wildcard characters with LIKE:

Wildcard	Meaning	Example
%	Represents **zero or more characters**	'J%'
_	Represents **exactly one character**	'J_hn'

Find employees whose first name starts with J.

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE Fname LIKE 'J%';
```

J% means:

- The first character must be J
- The remaining characters can be anything

Find employees whose last name ends with g.

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE Lname LIKE '%g';
```

%g means:

- Any characters can appear before
- The last character must be **g**

Using Underscore (_) Wildcard

Find employees whose name has exactly 4 characters and starts with J.

```
SELECT Fname
FROM EMPLOYEE
WHERE Fname LIKE 'J___';
```

- _ represents exactly one character.
- First character = J
- Next 3 characters (any letters)

NOT LIKE Operator

NOT LIKE finds values that **do not match a pattern**.

Example:

Find employees whose names do not start with J.

```
SELECT Fname
FROM EMPLOYEE
WHERE Fname NOT LIKE 'J%';
```

Find the top 3 highest-paid employees whose names start with J.

```
SELECT Fname, Salary
FROM EMPLOYEE
WHERE Fname LIKE 'J%'
ORDER BY Salary DESC
LIMIT 3;
```

IN Operator

The **IN** operator is used to check whether a value matches **any value in a list or the result of a subquery**.

Syntax

```
SELECT column_name
FROM table_name
WHERE column_name IN (subquery);
```

Find first name and last name of employees in department no 5.

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE Dno IN
(
    SELECT Dnumber
    FROM DEPARTMENT
    WHERE Dnumber = 5
);
```

ANY Operator

The **ANY** operator returns **TRUE if the comparison is true for at least one value returned by the subquery**.

Syntax

```
SELECT column_name
FROM table_name
WHERE column_name operator ANY (subquery);
```

The operator must be a standard comparison operator (=, <>, !=, >, >=, <, or <=).

```
SELECT Fname, Salary
FROM EMPLOYEE
WHERE Salary > ANY
(
   SELECT Salary
   FROM EMPLOYEE
   WHERE Dno = 5
);
```

```
SELECT Salary
FROM EMPLOYEE
WHERE Dno = 5
```

Result Grid

Salary
30000.00
40000.00
25000.00
38000.00

ALL Operator

The **ALL** operator returns **TRUE only if the comparison is true for every value returned by the subquery**.

SELECT column_name

FROM table_name

WHERE column_name **operator** ALL (subquery);

The operator must be a standard comparison operator (=, <>, !=, >, >=, <, or <=).

```
SELECT Fname, Salary
FROM EMPLOYEE
WHERE Salary > ALL
(
    SELECT Salary
    FROM EMPLOYEE
    WHERE Dno = 5
);
```

EXISTS is a SQL operator used to **check whether a subquery returns any rows or not**.

- If the subquery returns **one or more rows**, then EXISTS is TRUE.
- If the subquery returns **no rows**, then EXISTS is FALSE.

Syntax

```
SELECT column_name
FROM table_name
WHERE EXISTS
(
    SELECT column_name
    FROM table_name
    WHERE condition
);
```

Find Departments that have Employees.

```
SELECT Dname
FROM DEPARTMENT D
WHERE EXISTS
(
    SELECT *
    FROM EMPLOYEE E
    WHERE E.Dno = D.Dnumber
);
```

NOT EXISTS is the opposite of EXISTS.

It checks whether a subquery returns **no rows**.

- No matching rows → TRUE
- Any matching rows → FALSE

Syntax

```
SELECT column_name
FROM table_name
WHERE NOT EXISTS
(
    SELECT column_name
    FROM table_name
    WHERE condition
);
```

```
SELECT Dname
FROM DEPARTMENT D
WHERE NOT EXISTS
(
    SELECT *
    FROM EMPLOYEE E
    WHERE E.Dno = D.Dnumber
);
```

Operator	Meaning
IN	Checks whether a value exists in a list or subquery result.
ANY	Returns TRUE if the condition is true for at least one value.
ALL	Returns TRUE if the condition is true for every value.
EXISTS	Returns TRUE if the subquery returns one or more rows.
NOT EXISTS	Returns TRUE if the subquery returns no rows.

- **EMPLOYEE** (Fname, Minit, Lname, **Ssn**, Bdate, Address, Sex, Salary, Super_ssn, **Dno**)
- **DEPARTMENT** (Dname, **Dnumber**, Mgr_ssn, Mgr_start_date)
- **PROJECT** (Pname, **Pnumber**, Plocation, **Dnum**)
- **WORKS_ON** (Essn, **Pno**, Hours)
- **DEPENDENT** (**Essn**, Dependent_name, Sex, Bdate, Relationship)

1. Display all employees.

```
SELECT *
FROM EMPLOYEE;
```

2. Display employee first name, last name and salary.

```
SELECT Fname, Lname, Salary
FROM EMPLOYEE;
```

3. Display Fname, Lname of employees who work in Department 5.

```
SELECT Fname, Lname
FROM EMPLOYEE
WHERE Dno = 5;
```

4. Find Fname, Lname and salary of employees whose salary is greater than 40000.

```
SELECT Fname, Lname, Salary
FROM EMPLOYEE
WHERE Salary > 40000;
```

5. Display Fname, Lname and salary of employees sorted by salary from highest to lowest.

```
SELECT Fname, Lname, Salary
FROM EMPLOYEE
ORDER BY Salary DESC;
```

6. Display Fname, Lname and salary of employees sorted by salary from lowest to highest.

```
SELECT Fname, Lname, Salary
FROM EMPLOYEE
ORDER BY Salary ASC;
```

7. Find the maximum salary.

```
SELECT MAX(Salary) AS Highest_Salary
FROM EMPLOYEE;
```

8. Find the minimum salary.

```
SELECT MIN(Salary) AS Lowest_Salary
FROM EMPLOYEE;
```

9. Find the total salary.

```
select sum(salary) as Total_Salary
from employee;
```

10. How many employees are there in the company?

```
SELECT COUNT(*)
FROM EMPLOYEE;
```

11. Find the average salary.

```
select avg(salary) As Avg_Salary
from employee;
```

- Calculate the total number of employees in the company.

```
SELECT COUNT(*) AS Total_Employees
FROM EMPLOYEE;
```

- Calculate the average salary of all employees.

```
SELECT AVG(Salary) AS Average_Salary
FROM EMPLOYEE;
```

- Find the highest salary in the company.

```
SELECT MAX(Salary) AS Highest_Salary
FROM EMPLOYEE;
```

- Find the lowest salary in the company.

```
SELECT MIN(Salary) AS Lowest_Salary
FROM EMPLOYEE;
```

- Calculate the total salary paid to all employees.

```
SELECT SUM(Salary) AS Total_Salary
FROM EMPLOYEE;
```

- Calculate the number of employees in each department.

```
SELECT Dno, COUNT(*) AS Total_Employees
FROM EMPLOYEE
GROUP BY Dno;
```

- Calculate the average salary for each department.

```
SELECT Dno, AVG(Salary) AS Average_Salary
FROM EMPLOYEE
GROUP BY Dno;
```

- Display departments that have more than 3 employees.

```
SELECT Dno, COUNT(*) AS Total_Employees
FROM EMPLOYEE
GROUP BY Dno
HAVING COUNT(*) > 3;
```

- Calculate the total number of hours worked on each project.

```
SELECT Pno, SUM(Hours) AS Total_Hours
FROM WORKS_ON
GROUP BY Pno;
```

- Calculate the average number of hours worked on each project.

```
SELECT Pno, AVG(Hours) AS Average_Hours
FROM WORKS_ON
GROUP BY Pno;
```

- Find the maximum number of hours worked on each project.

```
SELECT Pno, MAX(Hours) AS Maximum_Hours
FROM WORKS_ON
GROUP BY Pno;
```

- Display each department name and the number of employees working in it.

```
SELECT D.Dname, COUNT(E.Ssn) AS Total_Employees
FROM DEPARTMENT D, EMPLOYEE E
WHERE D.Dnumber = E.Dno
GROUP BY D.Dname;
```

- Display each department name and the average salary of its employees.

```
SELECT D.Dname, AVG(E.Salary) AS Average_Salary
FROM DEPARTMENT D, EMPLOYEE E
WHERE D.Dnumber = E.Dno
GROUP BY D.Dname;
```

- Find the department whose employees have the highest average salary.

```
SELECT D.Dname, AVG(E.Salary) AS Average_Salary
FROM DEPARTMENT D, EMPLOYEE E
WHERE D.Dnumber = E.Dno
GROUP BY D.Dname
ORDER BY Average_Salary DESC;
```

- Display projects whose total working hours are greater than 100.

```
SELECT Pno, SUM(Hours) AS Total_Hours
FROM WORKS_ON
GROUP BY Pno
HAVING SUM(Hours) > 100;
```

- Find the highest average salary among all departments.

```
SELECT D.Dname, AVG(E.Salary) AS Average_Salary
FROM DEPARTMENT D JOIN EMPLOYEE E
ON D.Dnumber = E.Dno
GROUP BY D.Dname
ORDER BY Average_Salary DESC
LIMIT 1;
```

```
SELECT D.Dname, AVG(E.Salary) AS Average_Salary
FROM DEPARTMENT D JOIN EMPLOYEE E
ON D.Dnumber = E.Dno
GROUP BY D.Dname
HAVING AVG(E.Salary) >= ALL
(
    SELECT AVG(Salary)
    FROM EMPLOYEE
    GROUP BY Dno
);
```

- Find employees who work more hours than any employee working on Project 1.

```
SELECT Essn, Hours
FROM WORKS_ON
WHERE Hours > ANY
(
   SELECT Hours
   FROM WORKS_ON
   WHERE Pno = 1
);
```

- Find employees whose salary is greater than any employee salary in Department 5.

```
SELECT Fname, Salary
FROM EMPLOYEE
WHERE Salary > ANY
(
   SELECT Salary
   FROM EMPLOYEE
   WHERE Dno = 5
);
```

- Find employees whose salary is greater than all employees in Department 5.

```
SELECT Fname, Salary
FROM EMPLOYEE
WHERE Salary > ALL
(
    SELECT Salary
    FROM EMPLOYEE
    WHERE Dno = 5
);
```